



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

开课学期: 2025 春季
课程名称: 人工智能 (实验)
实验名称: Pacman
实验性质: 综合设计型
实验学时: 4 地点: T2507
学生班级: 5
学生学号: 220110515
学生姓名: 金正达
评阅教师: _____
报告成绩: _____

实验与创新实践教育中心制

2025 年 3 月

1、PositionSearchProblem: 问题 1-4

1.1 代码实现

深度优先搜索:

```
def depthFirstSearch(problem: SearchProblem):
    """
    Search the deepest nodes in the search tree first.

    Your search algorithm needs to return a list of actions that reaches the
    goal. Make sure to implement a graph search algorithm.

    To get started, you might want to try some of these simple commands to
    understand the search problem that is being passed in:

    print("Start:", problem.getStartState())
    print("Is the start a goal?", problem.isGoalState(problem.getStartState()))
    print("Start's successors:", problem.getSuccessors(problem.getStartState()))
    """
    "*** YOUR CODE HERE ***"
    stack = util.Stack()
    visited = set()
    stack.push((problem.getStartState(), []))
    while not stack.isEmpty():
        state, actions = stack.pop()
        if problem.isGoalState(state):
            return actions
        if state not in visited:
            visited.add(state)
            for successor, action, _ in problem.getSuccessors(state):
                stack.push((successor, actions + [action]))
    return []
```

广度优先搜索:

```
def breadthFirstSearch(problem: SearchProblem):
    """Search the shallowest nodes in the search tree first."""
    "*** YOUR CODE HERE ***"
    queue = util.Queue()
    visited = set()
    queue.push((problem.getStartState(), []))
    while not queue.isEmpty():
        state, actions = queue.pop()
        if problem.isGoalState(state):
            return actions
        if state not in visited:
            visited.add(state)
            for successor, action, _ in problem.getSuccessors(state):
                queue.push((successor, actions + [action]))
    return []
```

代价一致性搜索：

```
def uniformCostSearch(problem: SearchProblem):
    """Search the node of least total cost first."""
    """*** YOUR CODE HERE ***"""
    priorityQueue = util.PriorityQueue()
    visited = set()
    priorityQueue.push((problem.getStartState(), []), 0)
    while not priorityQueue.isEmpty():
        state, actions = priorityQueue.pop()
        if problem.isGoalState(state):
            return actions
        if state not in visited:
            visited.add(state)
            for successor, action, stepCost in problem.getSuccessors(state):
                totalCost = problem.getCostOfActions(actions) + stepCost
                priorityQueue.push((successor, actions + [action]), totalCost)
    return []
```

A*搜索：

```
def aStarSearch(problem: SearchProblem, heuristic=nullHeuristic):
    """Search the node that has the lowest combined cost and heuristic first."""
    """*** YOUR CODE HERE ***"""
    priorityQueue = util.PriorityQueue()
    visited = set()
    priorityQueue.push((problem.getStartState(), []), 0)
    while not priorityQueue.isEmpty():
        state, actions = priorityQueue.pop()
        if problem.isGoalState(state):
            return actions
        if state not in visited:
            visited.add(state)
            for successor, action, stepCost in problem.getSuccessors(state):
                totalCost = problem.getCostOfActions(actions) + stepCost + heuristic(successor, problem)
                priorityQueue.push((successor, actions + [action]), totalCost)
    return []
```

1.2 实验结果分析

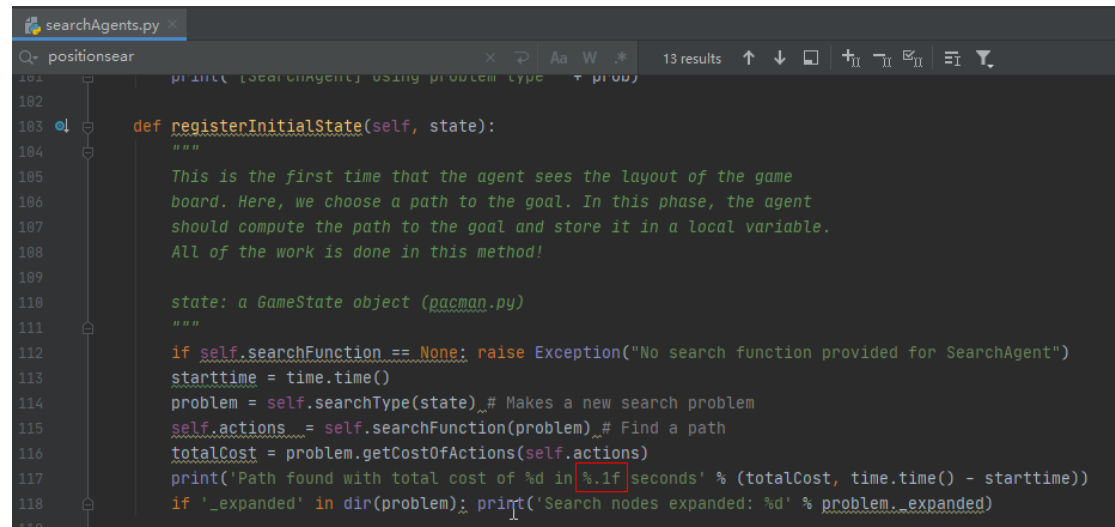
以下列表格的形式，列出四种算法在 bigMaze 迷宫上的实验数据，包括扩展节点数、路径 cost、耗时，以及在 pacman 项目中的完备性、代价最优性，并就完备性和代价最优性给出总结说明。

	扩展节点数	路径 cost	耗时/s	完备性	代价最优性
dfs	390	210	0.00982	是	否
bfs	620	210	0.01541	是	是
ucs	620	210	0.05306	是	是
A*	549	210	0.04676	是	是

完备性：当存在解时，算法能保证找到解，当不存在解时，能保证报告失败。

代价最优性：找到了所有解中路径代价最小的解。

关于耗时数据，需要改动计时的代码精确到小数点后 5 位，在 searchAgents.py 中找到图中所示代码，将红框中的%.1f 改为%.5f。



```
101 print([searchAgent] using problem type + prob)
102
103 def registerInitialState(self, state):
104     """
105     This is the first time that the agent sees the layout of the game
106     board. Here, we choose a path to the goal. In this phase, the agent
107     should compute the path to the goal and store it in a local variable.
108     All of the work is done in this method!
109
110     state: a GameState object (pacman.py)
111     """
112     if self.searchFunction == None: raise Exception("No search function provided for SearchAgent")
113     starttime = time.time()
114     problem = self.searchType(state) # Makes a new search problem
115     self.actions = self.searchFunction(problem) # Find a path
116     totalCost = problem.getCostOfActions(self.actions)
117     print('Path found with total cost of %d in %.1f seconds' % (totalCost, time.time() - starttime))
118     if '_expanded' in dir(problem): print('Search nodes expanded: %d' % problem._expanded)
119
```

2、CornersProblem: 问题 5-6

2.1 问题 5 的代码

```
def getStartState(self):
    """
    Returns the start state (in your state space, not the full Pacman state
    space)
    """
    "*** YOUR CODE HERE ***"
    return (self.startingPosition, (False, False, False, False))
```

```
def isGoalState(self, state: Any):
    """
    Returns whether this search state is a goal state of the problem.
    """
    "*** YOUR CODE HERE ***"
    return state[1] == (True, True, True, True)
```

```

def getSuccessors(self, state: Any):
    """
    Returns successor states, the actions they require, and a cost of 1.

    As noted in search.py:
    For a given state, this should return a list of triples, (successor,
    action, stepCost), where 'successor' is a successor to the current
    state, 'action' is the action required to get there, and 'stepCost'
    is the incremental cost of expanding to that successor
    """

    successors = []
    for action in [Directions.NORTH, Directions.SOUTH, Directions.EAST, Directions.WEST]:
        # Add a successor state to the successor list if the action is legal
        # Here's a code snippet for figuring out whether a new position hits a wall:
        #   x,y = currentPosition
        #   dx, dy = Actions.directionToVector(action)
        #   nextx, nexty = int(x + dx), int(y + dy)
        #   hitsWall = self.walls[nextx][nexty]

        x, y = state[0]
        dx, dy = Actions.directionToVector(action)
        nextx, nexty = int(x + dx), int(y + dy)
        hitsWall = self.walls[nextx][nexty]
        if not hitsWall:
            nextState = (nextx, nexty)
            cornersVisited = list(state[1])
            if nextState in self.corners:
                index = self.corners.index(nextState)
                cornersVisited[index] = True
            successors.append((nextState, tuple(cornersVisited), action, 1))

    self._expanded += 1 # DO NOT CHANGE
    return successors

```

2.2 问题 6 的启发函数设计和代码实现

尝试两种不同的启发函数实现

(1) 明确给出启发函数的公式，再给出代码截图，以及一致性、可纳性说明。

(2) 给出两个启发函数占优性的比较，即从理论上说明哪个启发函数更好。

方法一：

$$h_1(\text{state}) = (1 / N) * \sum [\text{manhattanDistance}(\text{position}, \text{corner}_i)]$$

```

position, cornersVisited = state
unvisitedCorners = [corners[i] for i in range(len(corners)) if not cornersVisited[i]]
if not unvisitedCorners:
    return 0

#####
distances = [util.manhattanDistance(position, corner) for corner in unvisitedCorners]
total_distance1 = sum(distances) // len(distances)

```

可纳性:

剩余 N 个角所需的实际路径成本, 与当前位置到这些角的曼哈顿距离的平均值之间没有直接的、保证小于等于的关系。考虑一种极端情况: 一个角很近, 其他角很远。实际路径至少要走完到最远角的距离, 而平均距离可能远小于这个最远距离, 但也可能在某些复杂情况下大于实际最优路径 (虽然曼哈顿距离忽略了墙壁, 但平均化这个操作使得它不一定是实际路径成本的下界)。因此, h_1 不保证是可纳的。

一致性:

h_1 不保证可纳, 它通常也不具备一致性。状态转移 (移动一步) 后, 未访问的角集合可能不变, 也可能减少一个。平均值的变化与单步成本 ($=1$) 之间的关系不满足一致性要求。

方法二:

$h_2(\text{state}) = \max [\text{manhattanDistance}(\text{position}, \text{corner}_i)]$

```
distances = [util.manhattanDistance(position, corner) for corner in unvisitedCorners]
maxDistance = max(distances)
total_distance2 = maxDistance
```

可纳性:

要访问所有剩余的角, 至少需要移动到那个距离当前位置最远的未访问角。到最远角的曼哈顿距离 $h_2(\text{state})$ 必然小于或等于实际到达所有剩余角所需的最短路径成本。 h_2 是可纳的。

一致性:

- 曼哈顿距离本身是一致的。
- 当应用于 CornersProblem 时, 目标是访问一组点。 h_2 取的是到未访问点集合中最远点的曼哈顿距离。当 Pacman 移动一步从 pos 到 pos' 时, $\text{manhattan}(\text{pos}, \text{farthest_corner}) \leq 1 + \text{manhattan}(\text{pos}', \text{farthest_corner})$ 。
- 如果 pos' 不是一个角, 或者是一个已经访问过的角, 那么未访问角集合不变, $h(n') = \max[\text{manhattan}(\text{pos}', \text{corner}_i)]$ 。由于 $\text{manhattan}(\text{pos}, \text{corner}_i) \leq 1 + \text{manhattan}(\text{pos}', \text{corner}_i)$ 对所有 corner_i 成立, 所以 $\max[\text{manhattan}(\text{pos}, \text{corner}_i)] \leq 1 + \max[\text{manhattan}(\text{pos}', \text{corner}_i)]$, 即 $h(n) \leq 1 + h(n')$ 。
- 如果 pos' 是一个之前未访问的角 c , 那么 n' 的未访问角集合比 n 少一个 c 。 $h(n')$ 是 pos' 到 剩余 未访问角的最大曼哈顿距离。 $h(n)$ 是 pos 到 所有 未访问角 (包括 c) 的最大曼哈顿距离。一致性要求 $h(n) \leq 1 + h(n')$ 仍然成立, 因为曼哈顿距离的三角不等式特性保证了这一点。 h_2 是一致的。

2.3 实验结果分析

以表格的形式列出两种启发函数在 mediumCorners 上的实验数据, 包括扩展节点数、路径 cost、耗时, 并说明哪个启发函数更好。

	扩展节点数	路径 cost	耗时
方法一	1318	106	0.5056
方法二	1136	106	0.4167

方法二更好, 耗时更短, 扩展节点数更少, 并且满足可纳性和一致性。

3、FoodSearchProblem: 问题 7-8

3.1 问题 7 的启发函数设计和代码

明确给出启发函数的公式, 再给出代码截图, 以及一致性、可纳性说明。

$h(\text{state}) = \max [\text{manhattanDistance}(\text{position}, \text{food})]$

```
position, foodGrid = state
"*** YOUR CODE HERE ***"
position, foodGrid = state
foodList = foodGrid.asList()

if not foodList:
    return 0

distances = [util.manhattanDistance(position, food) for food in foodList]
maxDistance = max(distances)

return maxDistance
```

可纳性:

- 从当前位置 position 到达最远食物点 food_far 的实际最短路径长度 (考虑墙壁) 必然大于或等于它们之间的曼哈顿距离 util.manhattanDistance(position, food_far) (因为曼哈顿距离忽略了墙壁, 是直线距离的下界)。
- 而吃掉所有食物的实际最短路径长度 h*(n), 必然大于或等于仅仅到达那个最远食物点 food_far 的实际最短路径长度。

该函数是可纳的。

一致性:

- 曼哈顿距离本身满足一致性条件（移动一步，到目标的曼哈顿距离减少量不会超过 1），而该启发函数取的是一组曼哈顿距离中的最大值，这个最大值相对于下一步状态的最大值也满足一致性要求。

3.2 问题 8 的代码实现

```
def isGoalState(self, state: Tuple[int, int]):  
    """  
    The state is Pacman's position. Fill this in with a goal test that will  
    complete the problem definition.  
    """  
    x,y = state  
  
    "*** YOUR CODE HERE ***"  
    # Check if the current position is a food position  
    return self.food[x][y]
```

```
def findPathToClosestDot(self, gameState: pacman.GameState):  
    """  
    Returns a path (a list of actions) to the closest dot, starting from  
    gameState.  
    """  
    # Here are some useful elements of the startState  
    startPosition = gameState.getPacmanPosition()  
    food = gameState.getFood()  
    walls = gameState.getWalls()  
    problem = AnyFoodSearchProblem(gameState)  
  
    "*** YOUR CODE HERE ***"  
    path = search.breadthFirstSearch(problem)  
    return path
```

4、总结

4.1 碰到的问题

在实现 CornersProblem 时，如何设计状态空间以便有效记录已访问的角落是一个难点。最初只用 Pacman 的位置作为状态，导致无法判断哪些角落已经访问，后来通过元组记录四个角落的访问情况，才保证了状态的唯一性和正确性。

在实现 cornersHeuristic 和 foodHeuristic 时，如何保证启发式函数既可行又

满足一致性和可采纳性是一个挑战。刚开始直接用所有未访问角落的曼哈顿距离之和，发现不够紧，后来改用当前位置到最远角落的距离，效果更好。

4.2 总结、建议

通过本次实验，深刻体会到合理设计状态空间对于搜索问题的效率和正确性至关重要。尤其是在涉及多个目标（如四个角落或所有食物）的情况下，状态不仅仅是位置，还要包含目标的访问情况。实现了 DFS、BFS、UCS 和 A 等算法，对它们的优缺点和适用场景有了更直观的理解。特别是 A 结合启发式函数后，能显著提升搜索效率。学会了如何设计和评估启发式函数，理解了启发式的可采纳性和一致性对 A*算法正确性的影响。

建议实验前可以更多讲述代码框架是怎么搭建的。